Skill – Week5

Session 9:

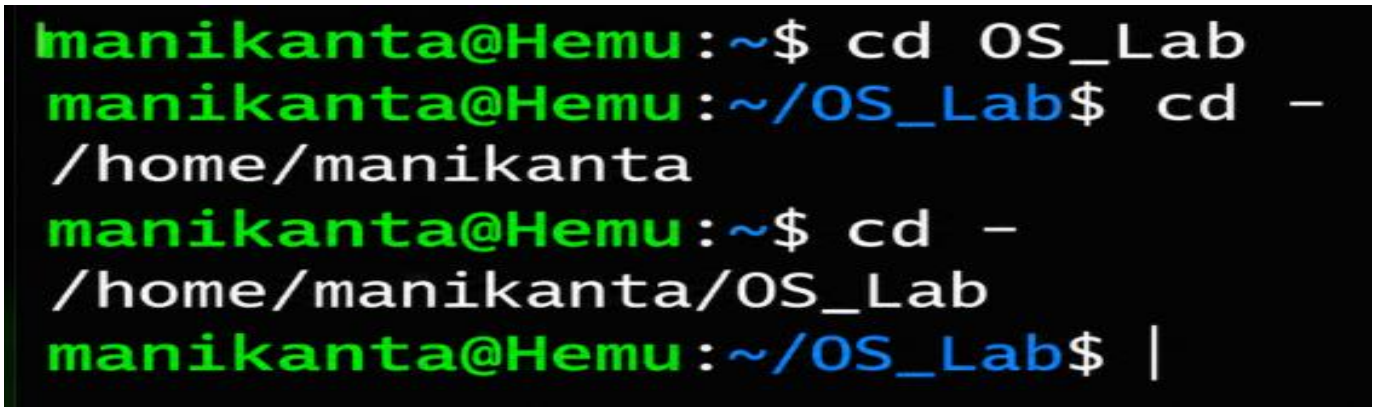
1. To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

```
manikanta@Hemu:~$ pwd
/home/manikanta
manikanta@Hemu:~$ cd OSSP
manikanta@Hemu:~/OSSP$ pwd
/home/manikanta/OSSP
manikanta@Hemu:~/OSSP$ cd ..
manikanta@Hemu:~$ pwd
/home/manikanta
```

```
manikanta@Hemu:~$ [ -d "OSSP" ] && echo "Directory exists" || echo "Directory does not exist"
Directory exists
```

```
manikanta@Hemu:~$ cd OSSP || echo "Error: Cannot change directory"
Error: Cannot change directory
manikanta@Hemu:~/OSSP$ |
```

```
manikanta@Hemu:~$ if cd OSSP
then
    echo "Directory changed successfully"
else
    echo "Error: Cannot change directory"
fi
Directory changed successfully
manikanta@Hemu:~/OSSP$ cd
manikanta@Hemu:~$ cd OSSP
manikanta@Hemu:~/OSSP$ pwd
/home/manikanta/OSSP
manikanta@Hemu:~/OSSP$ |
```


manikanta@Hemu:~$ cd OS_Lab
manikanta@Hemu:~/OS_Lab$ cd -
/home/manikanta
manikanta@Hemu:~$ cd -
/home/manikanta/OS_Lab
manikanta@Hemu:~/OS_Lab$

```
  GNU nano 8.7.1                          navigate.sh *
#!/bin/bash

echo "Current directory:"
pwd

echo "Enter directory path:"
read path

if [ -d "$path" ]; then

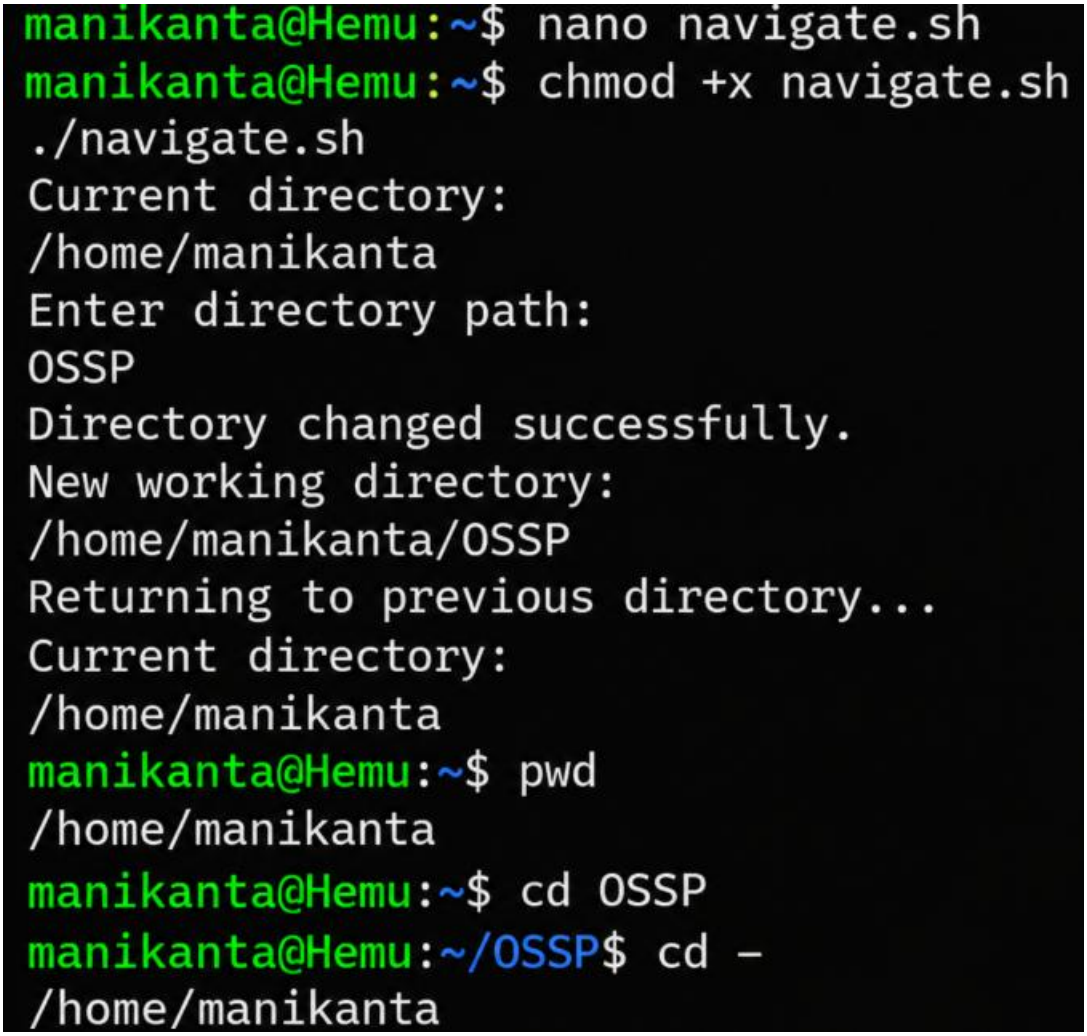
    olddir=$(pwd)

    if cd "$path"; then
        echo "Directory changed successfully."
        echo "New working directory:"
        pwd

        echo "Returning to previous directory..."
        cd "$olddir"

        echo "Current directory:"
        pwd
    else
        echo "Error: Cannot change directory."
    fi

else
    echo "Error: Directory does not exist."
fi
```



Report: The program successfully performs directory navigation by changing and validating the given path. Commands like cd are used to change the directory and pwd can be used to check the current working directory. It also handles invalid paths and keeps track of the previous directory, making the navigation process simple and reliable.

2. To Identify Built-ins, Create Dispatch Table, Execute In Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

// Function declarations
void builtin_cd(char *path);
void builtin_pwd();
void builtin_echo(char *text);
void builtin_exit();

// Built-in command structure
typedef void (*builtin_func)(char *);

typedef struct {
    char *name;
    builtin_func function;
} Builtin;

// Function to change directory
void builtin_cd(char *path) {
    if (path == NULL) {
        printf("cd: missing argument\n");
        return;
    }

    if (chdir(path) == 0) {
        printf("Directory changed successfully\n");
    } else {
        perror("cd");
    }
}

// Function to display current directory
void builtin_pwd() {
    char cwd[1024];
```

```c
// Function to display text
void builtin_echo(char *text) {
    if (text == NULL) {
        printf("\n");
    } else {
        printf("%s\n", text);
    }
}

// Function to exit shell
void builtin_exit() {
    printf("Exiting shell...\n");
    exit(0);
}

// Dispatch table
Builtin builtins[] = {
    {"cd", (builtin_func) builtin_cd},
    {"pwd", (builtin_func) builtin_pwd},
    {"echo", (builtin_func) builtin_echo},
    {"exit", (builtin_func) builtin_exit}
};

int builtin_count = sizeof(builtins) / sizeof(builtins[0]);

// Identify and execute built-in command
int execute_builtin(char *command, char *argument) {

    for (int i = 0; i < builtin_count; i++) {

        if (strcmp(command, builtins[i].name) == 0) {

            builtins[i].function(argument);

            return 1;
        }
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
// Function declarations from builtin.c
int execute_builtin(char *command, char *argument);
int main() {
    char input[1024];
    char *command;
    char *argument;
    printf("Simple Shell Started\n");
    while (1) {
        printf("shell> ");
        fflush(stdout);
        // Read command
        if (fgets(input, sizeof(input), stdin) == NULL) {
            break;
        }
        // Remove newline
        input[strcspn(input, "\n")] = '\0';
        // Skip empty command
        if (strlen(input) == 0) {
            continue;
        }
        // Extract command
        command = strtok(input, " ");
        // Extract argument
        argument = strtok(NULL, "");
        // Execute built-in command
        if (execute_builtin(command, argument)) {
            continue;
        }
        // Handle invalid command
        printf("Invalid command: %s\n", command);
    }
    return 0;
}
```

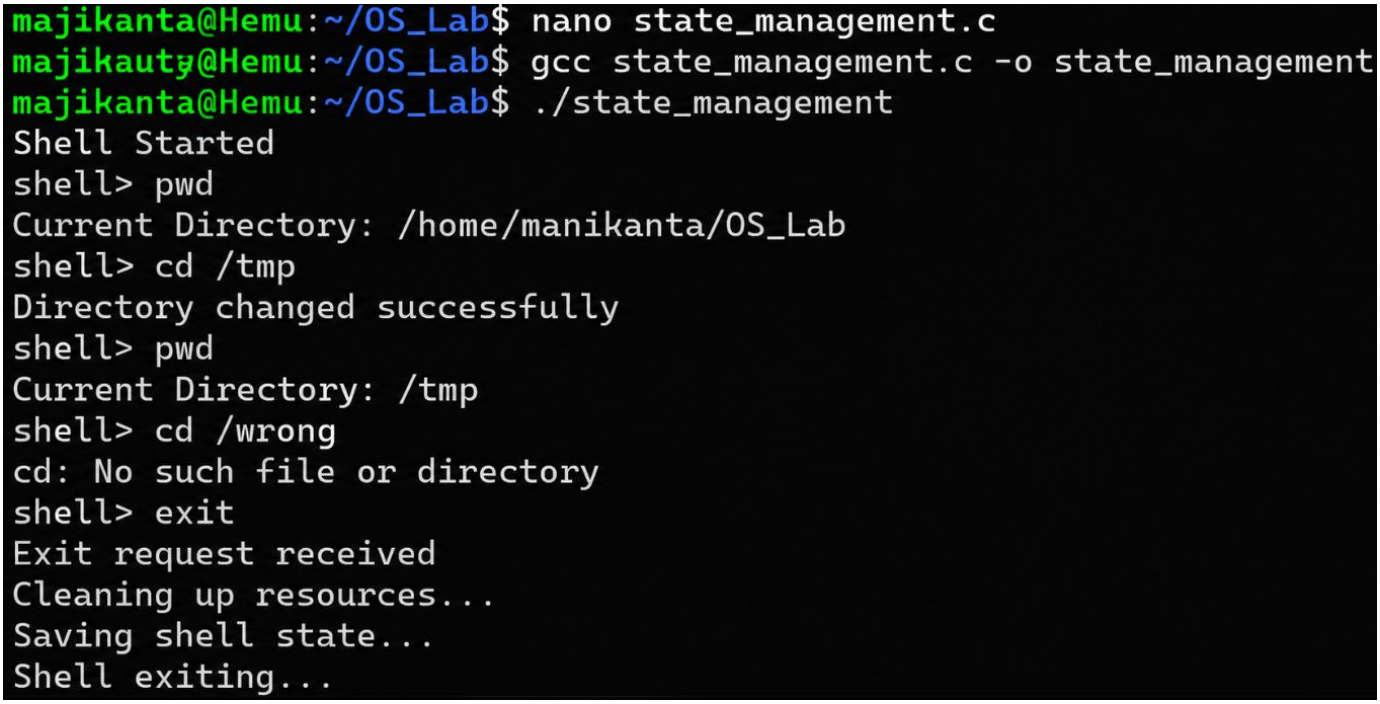
```
manikanta@Hemu:~/OS_Lab$ nano builtin.c
manikanta@Hemu:~/OS_Lab$ nano shell_dispatch.c
manikanta@Hemu:~/OS_Lab$ gcc shell_dispatch.c builtin.c -o shell_dispatch
manikanta@Hemu:~/OS_Lab$ ls
builtin.c    fork_example.c      shell_dispatch
fork_example fork_example.c.save shell_dispatch.c
manikanta@Hemu:~/OS_Lab$ ./shell_dispatch
Simple Shell Started
shell> pwd
/home/manikanta/OS_Lab
shell> echo Hello
Hello
shell> echo OperatingSystem Lab
OperatingSystem Lab
shell> cd /tmp
Directory changed successfully
shell> pwd
/tmp
shell> exit
Exiting shell...
manikanta@Hemu:~/OS_Lab$
```


Report: The program successfully identifies and executes built-in commands like cd and pwd using a dispatch table. It handles invalid commands and maintains shell state, making command execution simple and reliable.

Session 10:

1. To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
int main() {
    char cwd[1024];
    char input[100];
    printf("Shell Started\n");
    while (1) {
        printf("shell> ");
        fflush(stdout);
        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0';
        if (strcmp(input, "pwd") == 0) {
            if (getcwd(cwd, sizeof(cwd)) != NULL) {
                printf("Current Directory: %s\n", cwd);
            } else {
                perror("getcwd");
            }
        }
        else if (strncmp(input, "cd ", 3) == 0) {
            char *path = input + 3;
            if (chdir(path) == 0) {
                printf("Directory changed successfully\n");
            } else {
                perror("cd");
            }
        }
          else if (strcmp(input, "exit") == 0) {
           printf("Exit request received\n");
        printf("Cleaning up resources...\n");
            printf("Saving shell state...\n");
            printf("Shell exiting...\n");
            exit(0);
        }
        else {
           printf("Invalid command\n");
```



Report: The program successfully displays the current directory using pwd, changes directories using cd, and handles exit requests using exit. It also validates paths and maintains shell state, ensuring reliable operation.

2. To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>
// Validate variable name
int valid_name(char *name) {
    if (!isalpha(name[0]) && name[0] != '_')
        return 0;
    for (int i = 1; name[i] != '\0'; i++) {
        if (!isalnum(name[i]) && name[i] != '_')
            return 0;
    }
    return 1;
}
// Export environment variable
void export_variable(char *input) {
    char *equal = strchr(input, '=');
    if (equal == NULL) {
        printf("Invalid export syntax\n");
        return;
   }
    *equal = '\0';
    char *name = input;
    char *value = equal + 1;
    if (!valid_name(name)) {
        printf("Invalid variable name\n");
        return;
    }
    // Update environment variable
    if (setenv(name, value, 1) == 0) {
        printf("Exported: %s=%s\n", name, value);
    } else {
        perror("setenv");
    }
}
```

```
manikanta@Hemu:~/OS_Lab$ nano export_env.c
manikanta@Hemu:~/OS_Lab$ gcc export_env.c -o export_env
manikanta@Hemu:~/OS_Lab$ ./export_env
Environment Shell Started
shell> export NAME=Ajith
Exported: NAME=Ajith
shell> show NAME
NAME=Ajith
```

```
shell> export AGE=19
Exported: AGE=19
shell> export 123NAME=Hello
Invalid variable name
shell> export MY-NAME=Test
Invalid variable name
shell> export PATH_VAR=HelloChild
Exported: PATH_VAR=HelloChild
shell> child
Child process PATH_VAR=HelloChild
shell> hello
Invalid command
shell> exit
Exiting shell...
```

Report: The program successfully parses export commands, validates variable names, updates environment variables, and passes them to child processes. It also handles invalid commands and verifies the functionality through testing.